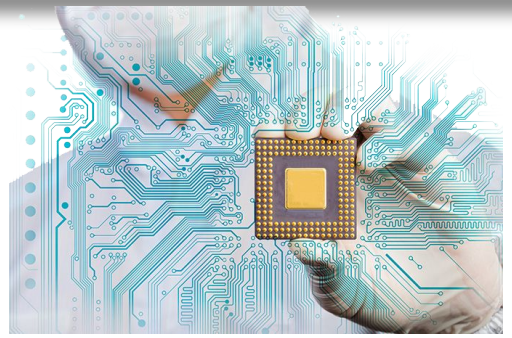
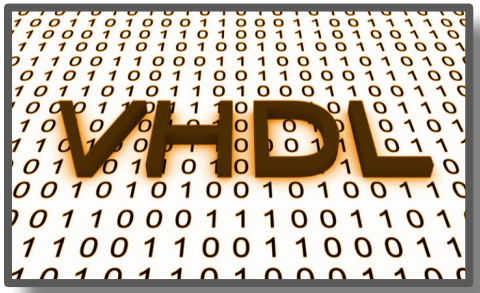
SUBPROGRAMAS EN VHDL

PRINCIPIO DE DISEÑO VLSI
DISEÑO DIGITAL CON TÉCNICAS DE ALTO NIVEL
ÁREA DISPOSITIVOS
DEPTO. DE ELECTRÓNICA Y COMPUTACIÓN
FI.UNMdP



SUBPROGRAMAS

- SUBPROGRAMAS
- FUNCIONES
- PROCEDIMIENTOS
- SOBRECARGA DE SUBPROGRAMAS



SE USAN PARA ESCRIBIR ALGORITMOS REUTILIZABLES Y AGRUPAR CÓDIGO

SUBPROGRAMAS

SE USAN PARA ESCRIBIR ALGORITMOS REUTILIZABLES Y AGRUPAR CÓDIGO. UN SUBPROGRAMA CONSTA DE:

- Parte declarativa, se definen estructuras de datos locales al subprograma.
- Sentencias secuenciales que describen lo que se debe realizar con los datos.

EN VHDL HAY DOS TIPOS DE SUBPROGRAMAS

- LAS FUNCIONES:
  - Retornan un único dato
  - Todos sus argumentos son entrada
- LOS PROCEDIMIENTOS:
  - Pueden devolver más de un valor
  - Los argumentos pueden ser de entrada, salida o entrada-salida

FUNCIONES

ESTÁN ORIENTADAS PARA REALIZAR ALGÚN TIPO DE CÁLCULO.

SINTAXIS:

```
[pure|impure ] function nombre_función [(lista_parámetros)] return
    tipo_retorno is
    {parte_declarativa} (No están permitidas señales, si variables)
begin
    {sentencias_secuenciales}
end [function] [nombre_función];
```

- Los parámetros formales son sólo del tipo in y la clase de objetos que pueden formar parte de ellos por defecto se asume que son constantes.
- Los tipos, subtipos, constantes y variables de la parte declarativa sólo tienen existencia cuando la función está activa y se inicializan cada vez que es invocada.
- Siempre debe haber al menos una sentencia return que es la que regresa el único valor del cálculo realizado por la función.
- En la parte de sentencias de la función no se pueden modificar variables o señales externas, tampoco puede aparecer ninguna sentencia wait.

FUNCIONES

EJEMPLO, STD_LOGIC A ENTERO:

```
use library ieee;
     use ieee.std_logic_1164.all;
entity convert is
     port(i1 : in std_logic_vector( 7 downto 0);
          o1 : out integer);
end convert;
architecture behave of convert is
     function vector_a_int(s : std_logic_vector( 7 downto 0))
          return integer is
          variable result : integer := 0;
     begin
          for i in 0 to 7 loop
               result := result * 2;
               if s(i) = '1' then
                    result := result + 1;
               end if;
          end loop;
          return result;
     end vector_a_int;
begin
     o1 <= vector_a_int(i1);
end behave;
```

FUNCIONES

EJEMPLO, STD_LOGIC A ENTERO:

- **UNA VEZ QUE SE HA DEFINIDO UNA FUNCIÓN SE PUEDE INVOCAR DESDE CUALQUIER PARTE DEL MODELO. PARA EL CASO ANTERIOR:**

```
process
    variable Entrada : std_logic_vector( 0 to 7 );
    variable Salida : natural;
begin
    ...
    Salida := vector_a_int ( Entrada );
    ...
end process;
```

- **TAMBIÉN SE LA PUEDE LLAMAR:**

```
Salida := vector_a_int(S => Entrada);
```

PROCEDIMIENTOS

- ESTÁN ORIENTADOS A REALIZAR CAMBIOS EN LOS DATOS, LOS QUE SE TRATAN INTERNAMENTE O A LOS QUE PUEDE ACCEDER.
  - SINTAXIS:

```
procedure nombre_procedimiento[(lista_parámetros )];

procedure nombre_procedimiento[(lista_parámetros)] is
    {parte_declarativa}
begin
    {sentencias_secuenciales}
end [procedure] [nombre_procedimiento];
```

- LOS PARÁMETROS PUEDEN SER IN, OUT O INOUT, POR OMISIÓN SE CONSIDERAN IN. LOS OBJETOS QUE SON PARTE DE LOS PARÁMETROS PUEDEN SER CONSTANTES, VARIABLES Y SEÑALES. POR OMISIÓN LOS PARÁMETROS IN SON CONSTANTES Y LOS OUT E INOUT SON VARIABLES.

PROCEDIMIENTOS

EJEMPLO: PROCEDIMIENTO QUE CONVIERTE UN DATO BIT_VECTOR, SIN SIGNO, EN SU NATURAL EQUIVALENTE.

```
procedure bv_a_natural(S : bit_vector(0 to 7); X : out natural) is
    variable Resultado : natural := 0;
begin
    for I in 0 to 7 loop
        Resultado := Resultado * 2 + bit'pos(S(I));
    end loop;
    X := Resultado;
end procedure bv_a_natural;
```

- UNA VEZ DEFINIDO SE LO PUEDE INVOCAR ASÍ:

```
process
    variable Entrada : bit_vector( 0 to 7 );
    variable Salida : natural;
begin
    ...
    bv_a_natural(Entrada,Salida);
    ...
end process;
```

- O ASÍ:

```
bv_a_natural(S => Entrada,X => Salida);
```

SOBRECARGA DE SUBPROGRAMAS

DOS PROGRAMAS ESTÁN SOBRECARGADOS CUANDO TIENEN COMO NOMBRE EL MISMO IDENTIFICADOR, PERO EL NÚMERO, TIPO Y ORDEN DE LOS PARÁMETROS ES DIFERENTE.

- EJEMPLO:

```
function Concatena ( a : string; b : string ) return string;
function Concatena ( a : character b : string ) return string;
function Concatena ( a : string; b : character ) return string;
```

- A LAS CUALES SE PUEDE ACCEDER:

```
Cadena := Concatena ( “abcde”, “fghij” );
Cadena := Concatena ( “abcdefghi”, ‘j’ );
Cadena := Concatena ( ‘a’, “bcdefghij” );
```

- SE PUEDEN ASIGNAR VALORES POR OMISIÓN:

```
procedure DespDerecha(Vector : inout bit_vector;Pos : bit_vector := b”0001”);
procedure DespDerecha(Vector : inout bit_vector; Pos : integer := 1);
```

SOBRECARGA DE OPERADORES

SE PUEDEN SOBRECARGAR LOS OPERADORES, ASÍ EL OPERADOR "=" PUEDE APLICARSE A CUALQUIER TIPO NUMÉRICO ENTERO, REAL O FÍSICO. HACE FALTA INDICAR LA FUNCIÓN QUE SE DESEA QUE REALICE EL OPERADOR.

- SE PUEDEN DEFINIR LAS OPERACIONES + Y AND SOBRE EL TIPO MITIPO:

```
fuction "+" ( a : MiTipo; b : MiTipo ) return MiTipo;
function "and" ( a : MiTipo; b : MiTipo ) return MiTipo;
```

- ESTAS FUNCIONES SE PUEDEN INVOCAR:

```
Var3 := Var1 + Var2;
Var4 := Var1 and Var2;
```

- O COMO LA LLAMADA TÍPICA A FUNCIÓN:

```
Var3 := "+"( Var1, Var2 );
Var4 := "and"( Var1, Var2 );
```